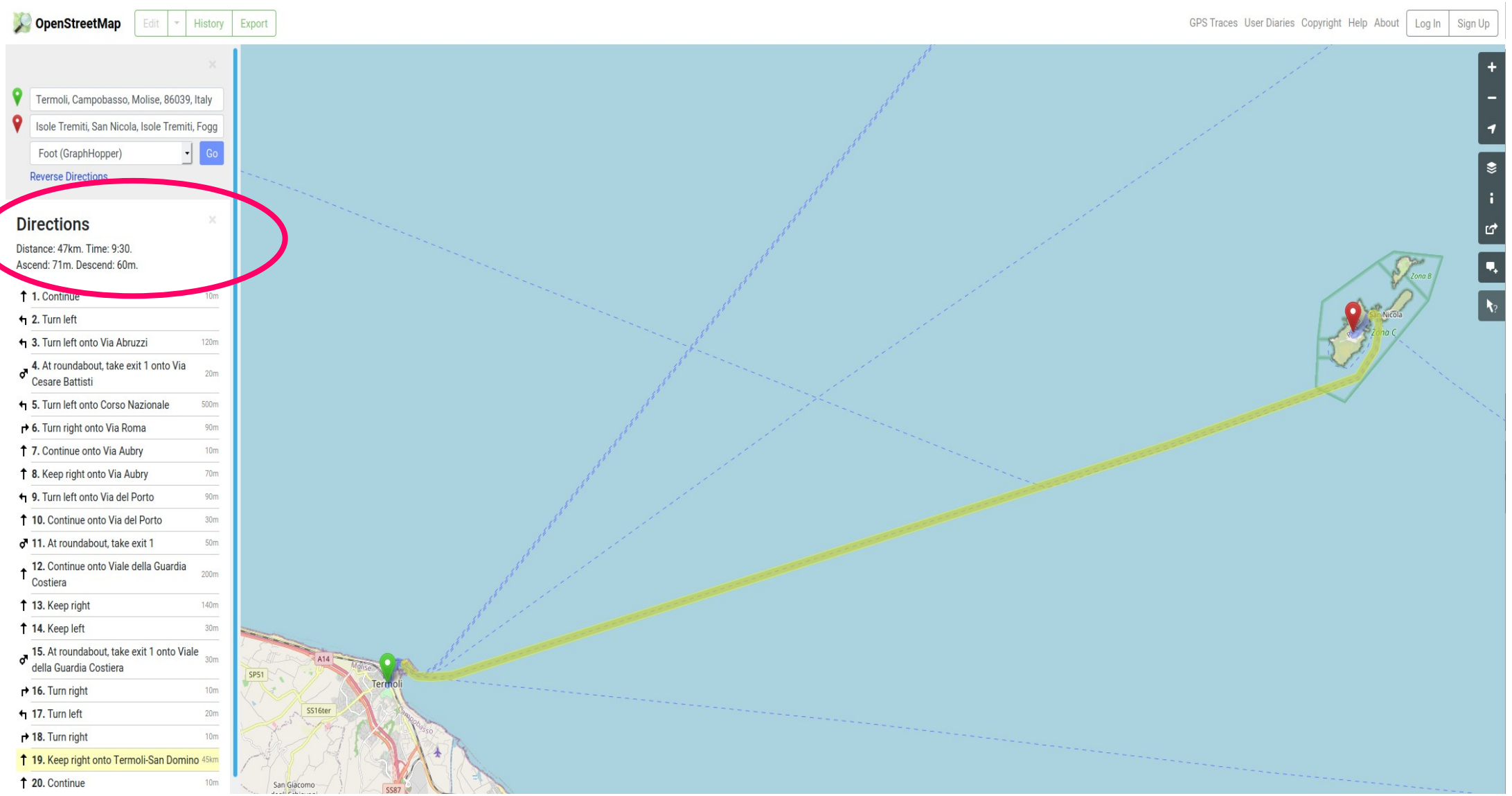
LEZIONE 5
introduzione e concetti generali

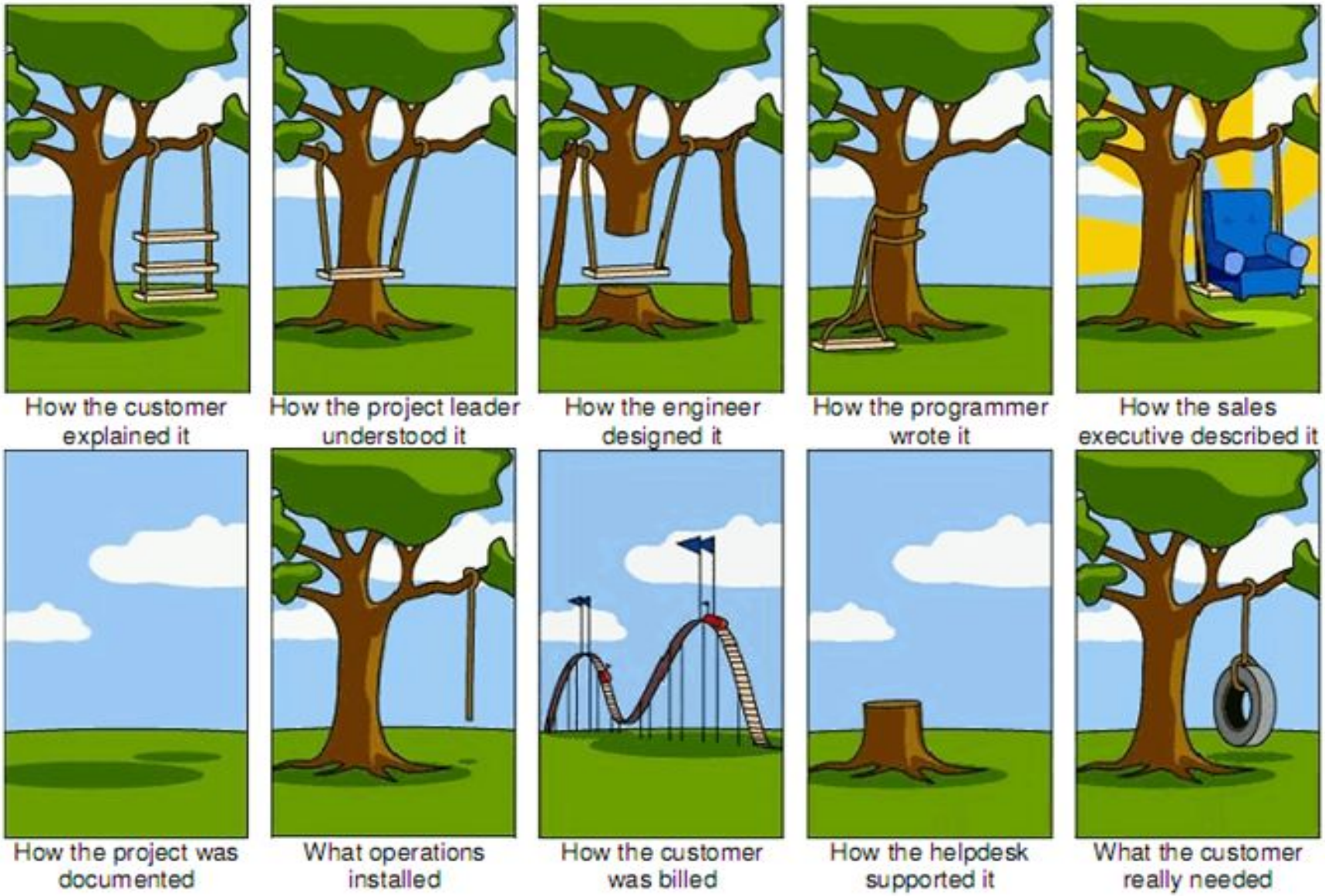
Ingegneria del Software 2 – Software Testing

Università degli Studi di Tor Vergata - Roma

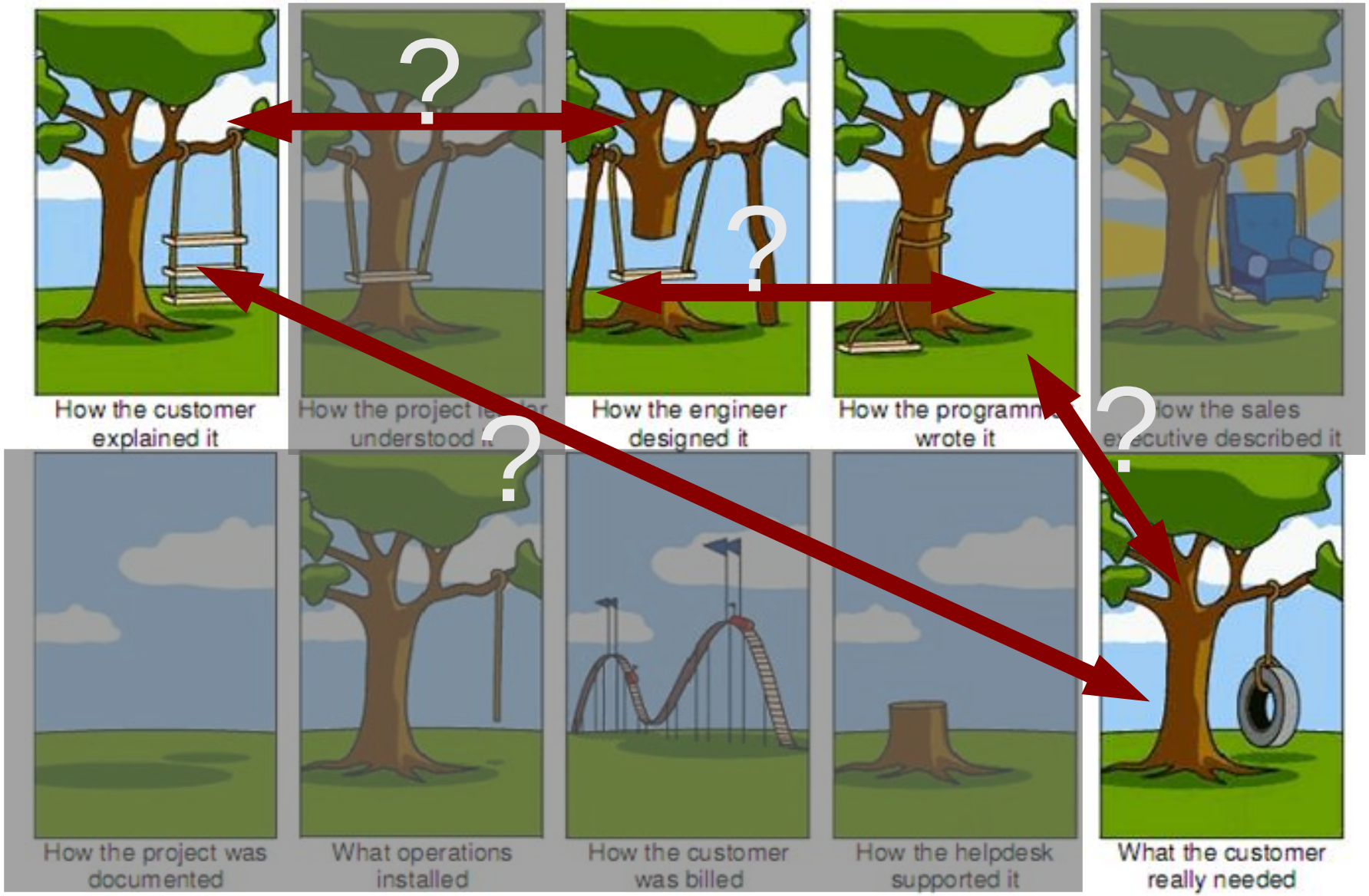
Guglielmo De Angelis
guglielmo.deangelis@iasi.cnr.it


OpenStreetMap
Edit
History
Export
GPS Traces
User Diaries
Copyright
Help
About
Log In
Sign Up
Termoli, Campobasso, Molise, 86039, Italy
Isole Tremiti, San Nicola, Isole Tremiti, Fogg
Foot (GraphHopper)
Go
Reverse Directions
Directions
Distance: 47km. Time: 9:30.
Ascend: 71m. Descend: 60m.
1. Continue 10m
2. Turn left
3. Turn left onto Via Abruzzi 120m
4. At roundabout, take exit 1 onto Via Cesare Battisti 20m
5. Turn left onto Corso Nazionale 500m
6. Turn right onto Via Roma 90m
7. Continue onto Via Aubry 10m
8. Keep right onto Via Aubry 70m
9. Turn left onto Via del Porto 90m
10. Continue onto Via del Porto 30m
11. At roundabout, take exit 1 50m
12. Continue onto Viale della Guardia Costiera 200m
13. Keep right 140m
14. Keep left 30m
15. At roundabout, take exit 1 onto Viale della Guardia Costiera 30m
16. Turn right 10m
17. Turn left 20m
18. Turn right 10m
19. Keep right onto Termoli-San Domino 45km
20. Continue 10m
A14
SP51
SS16ter
SS87
Molise
Termoli
Campobasso
San Giacomo
Zona B
San Nicola
Zona C

qualità e sistemi software



quali garanzie ?!?!?



citando Boehm

Verifica && Validazione:

- are we building the product right ?
  - il prodotto è conforme alle specifiche ?

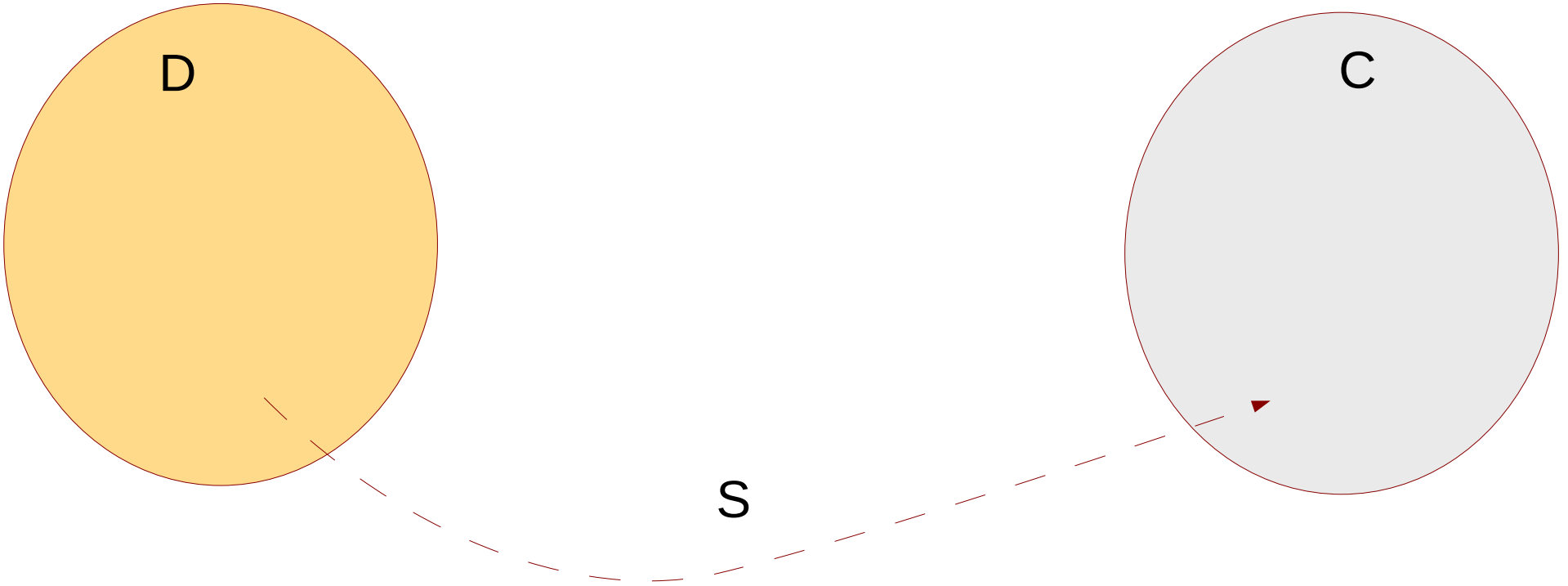
- are we building the right product ?
  - il prodotto è conforme alle attese del committente/utente ?

V&V

- tra gli obiettivi delle attività di V&V c'è rassicurare l’utilizzatore che il sistema è adatto allo scopo
- livello di fiducia che si intende fornire
  - in generale non è un concetto assoluto
  - è una livello di confidenza che può dipendere da diversi fattori:
    - funzione del software all’interno dell’organizzazione
    - attese dell’utente
    - politiche di mercato

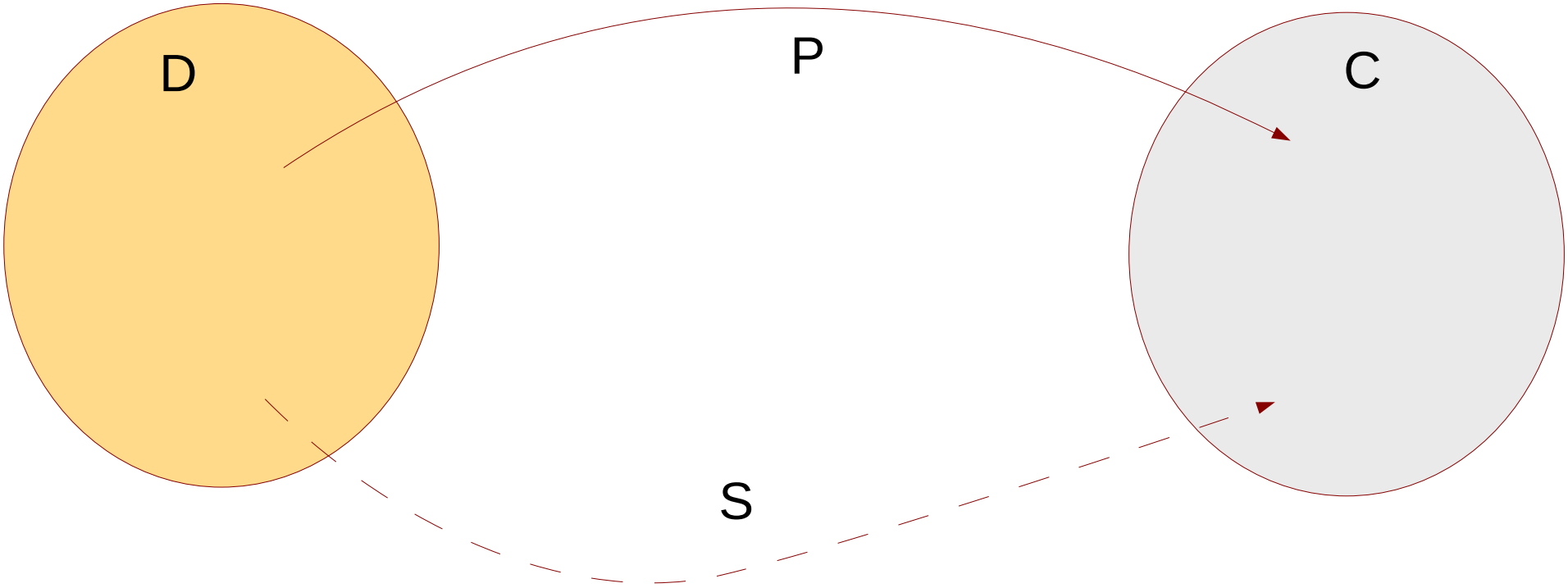
V&V – contesto generale

- sia S è una descrizione di un comportamento
  - cioè specifica da realizzare con un programma
  - una descrizione che dichiara per (classi su)gli input in D quali sono (le classi su)gli output attesi in C



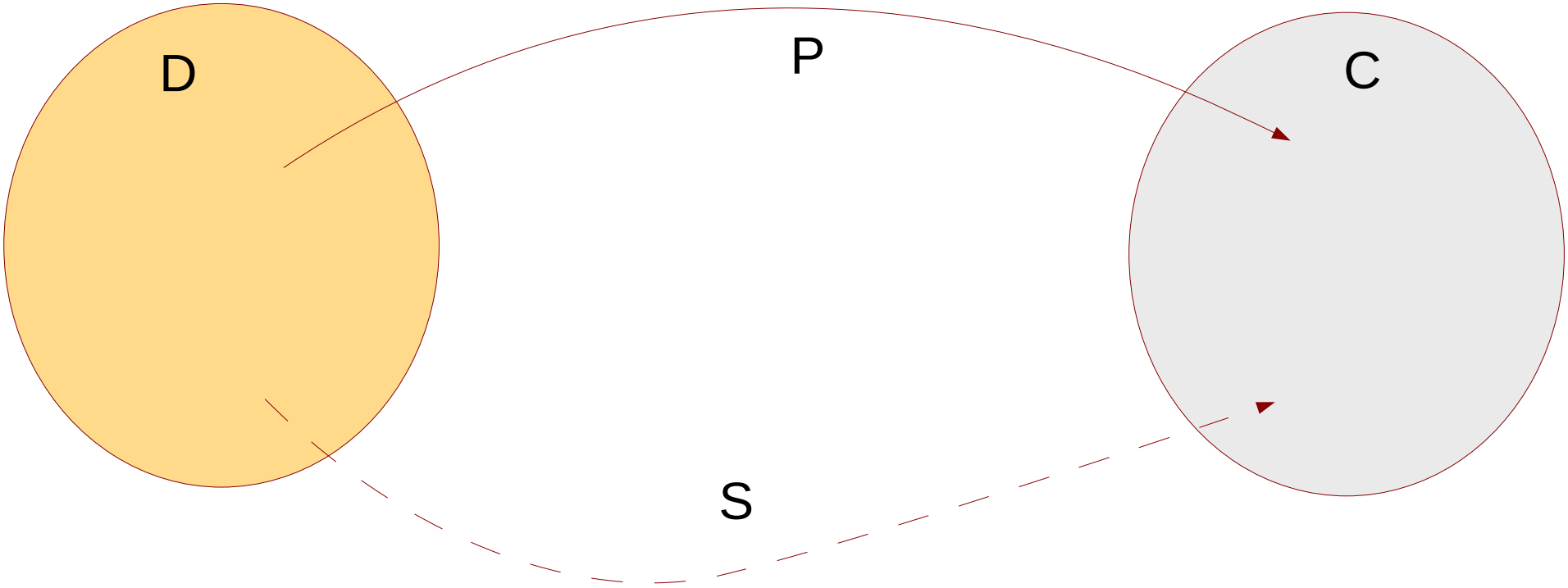
V&V – contesto generale

- sia S è una descrizione di un comportamento
  - cioè specifica da realizzare con un programma
  - una descrizione che dichiara per (classi su)gli input in D quali sono (le classi su)gli output attesi in C
- un programma P è una funzione da un dominio D ad un codominio C



V&V – contesto generale

- sia S è una descrizione di un comportamento
  - cioè specifica da realizzare con un programma
  - una descrizione che dichiara per (classi su)gli input in D quali sono (le classi su)gli output attesi in C
- un programma P è una funzione da un dominio D ad un codominio C
- P soddisfa la specifica S sse: ∀ d ∈ D, P(d) = S(d)



… differenti approcci per differenti obiettivi

- dimostrare che il programma P effettivamente soddisfa la specifica S su ogni input
  - verifica formale
- cercare di scoprire se il programma P nasconde qualche "*imperfezione*"
  - software testing and debugging

… differenti approcci per differenti obiettivi

- dimostrare che il programma P effettivamente soddisfa la specifica S su ogni input
  - verifica formale
- cercare di scoprire se il programma P nasconde qualche "*imperfezione*"
  - software testing and debugging

software testing vs debugging

come raggiungere l'adeguata confidenza di correttezza di un programma?

- pianificando strategie che evidenzino eventuali differenze tra P e S
  - ➔ software testing
- pianificando attività:
  - individuino la causa della difformità tra P ed S
  - rimuovano le anomalie in P non conformi ad S
    - ➔ debugging

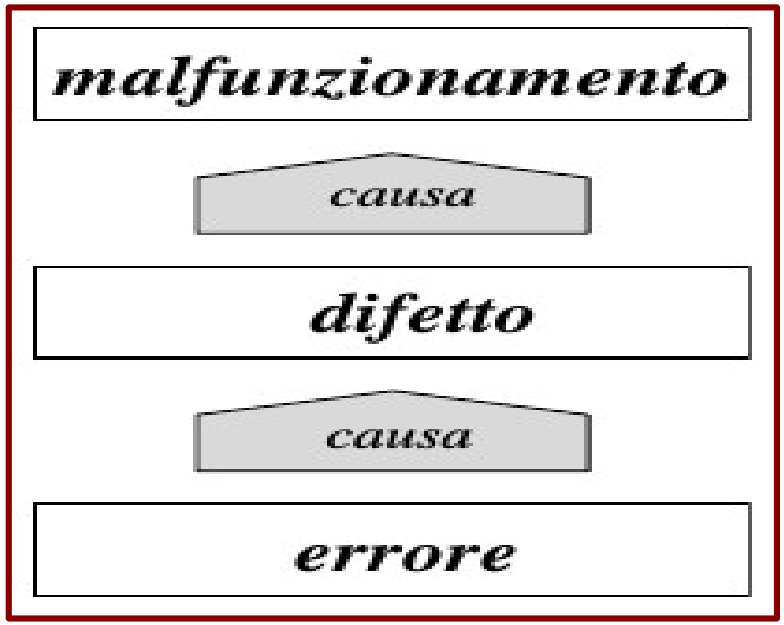
cosa significa :
CORRETTO ?!?!?

cosa significa : CORRETTO ?!?!?

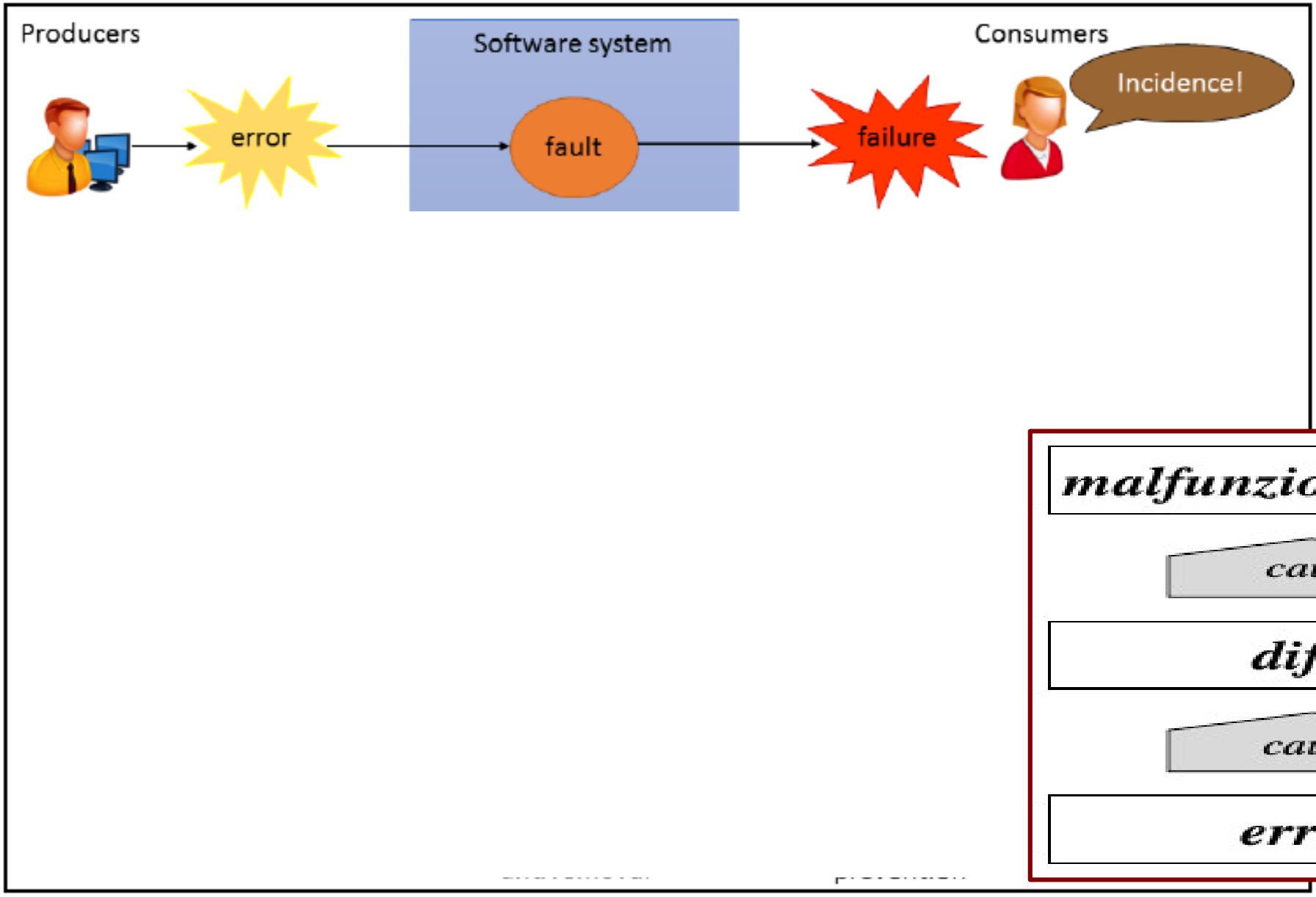
- APPROSSIMAZIONE BANALE DEL CONCETTO DI CORRETTEZZA :
  - “corretto” == “senza errori”

- cosa si intende per “errore” ?

qualche definizione (classica) – 1



qualche definizione (classica) – 2



malfunzionamento

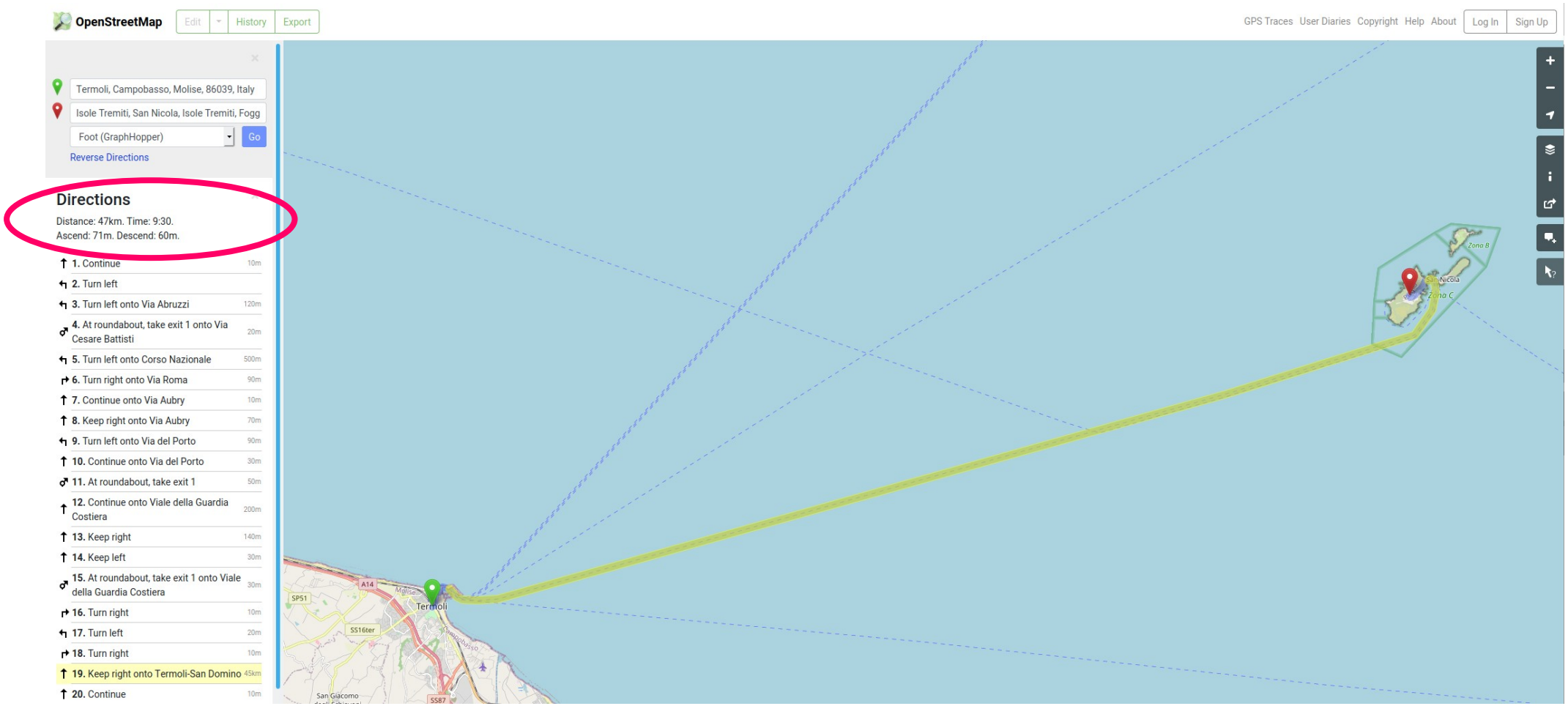
causa

difetto

causa

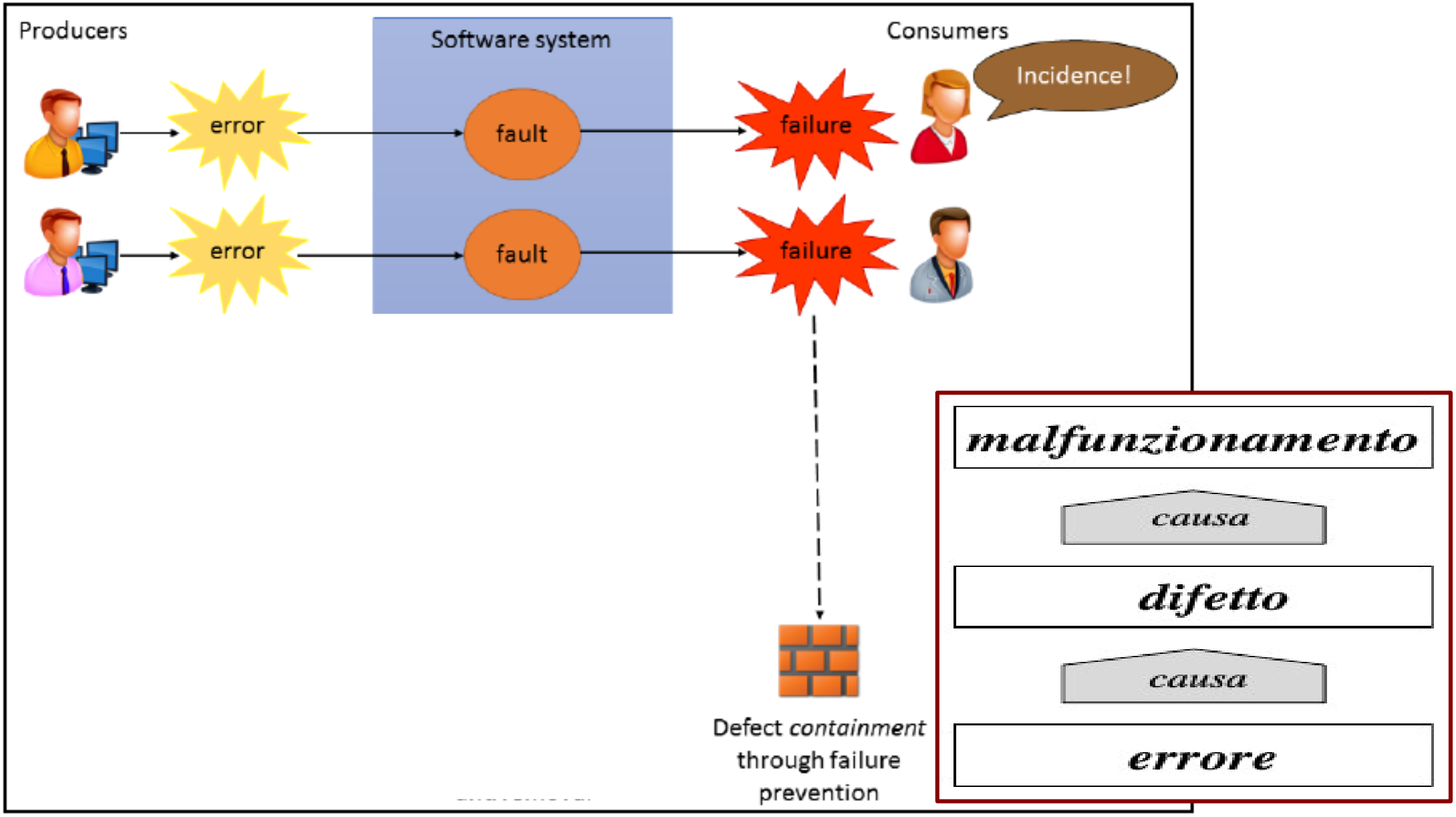
errore

failure

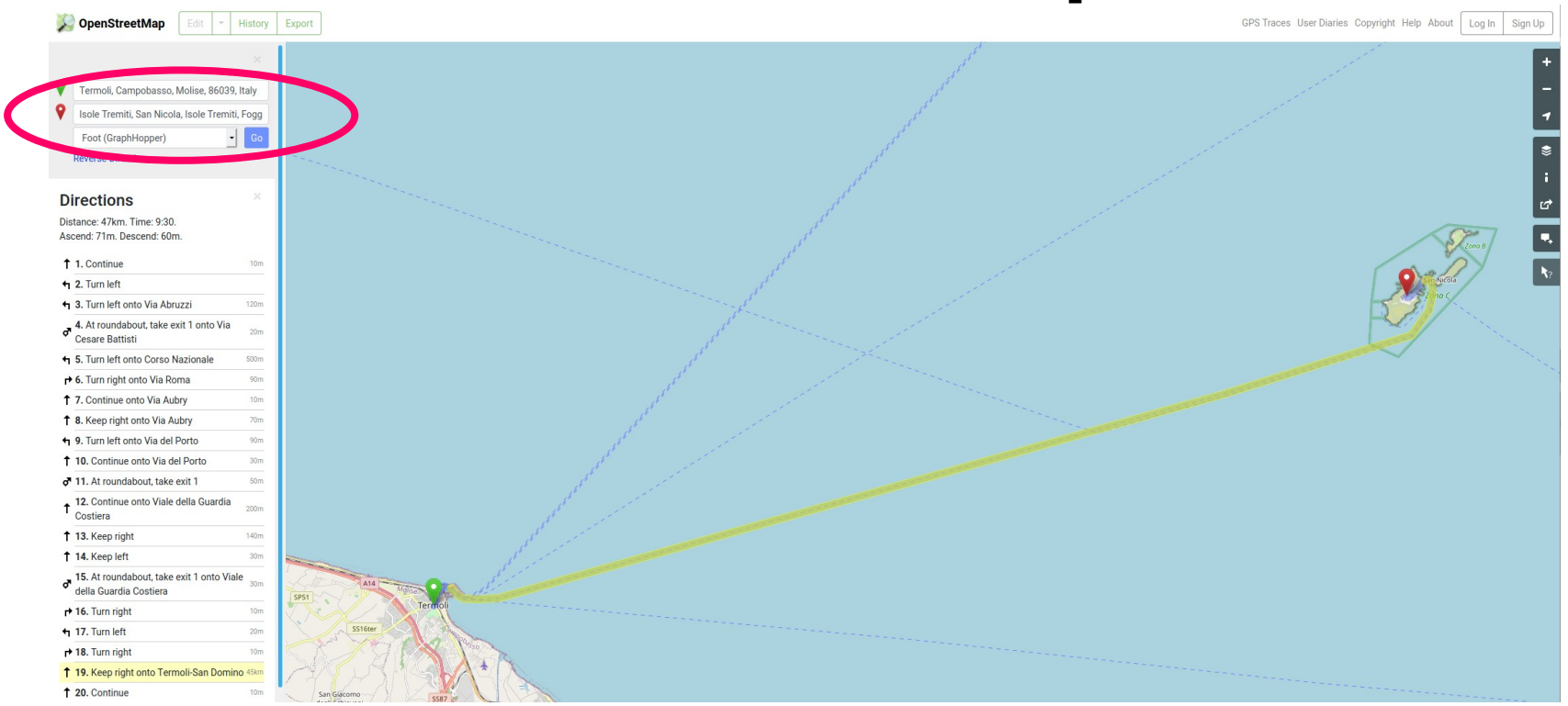


…. non posso camminare sott’acqua per raggiungere la mia destinazione!

qualche definizione (classica) – 2

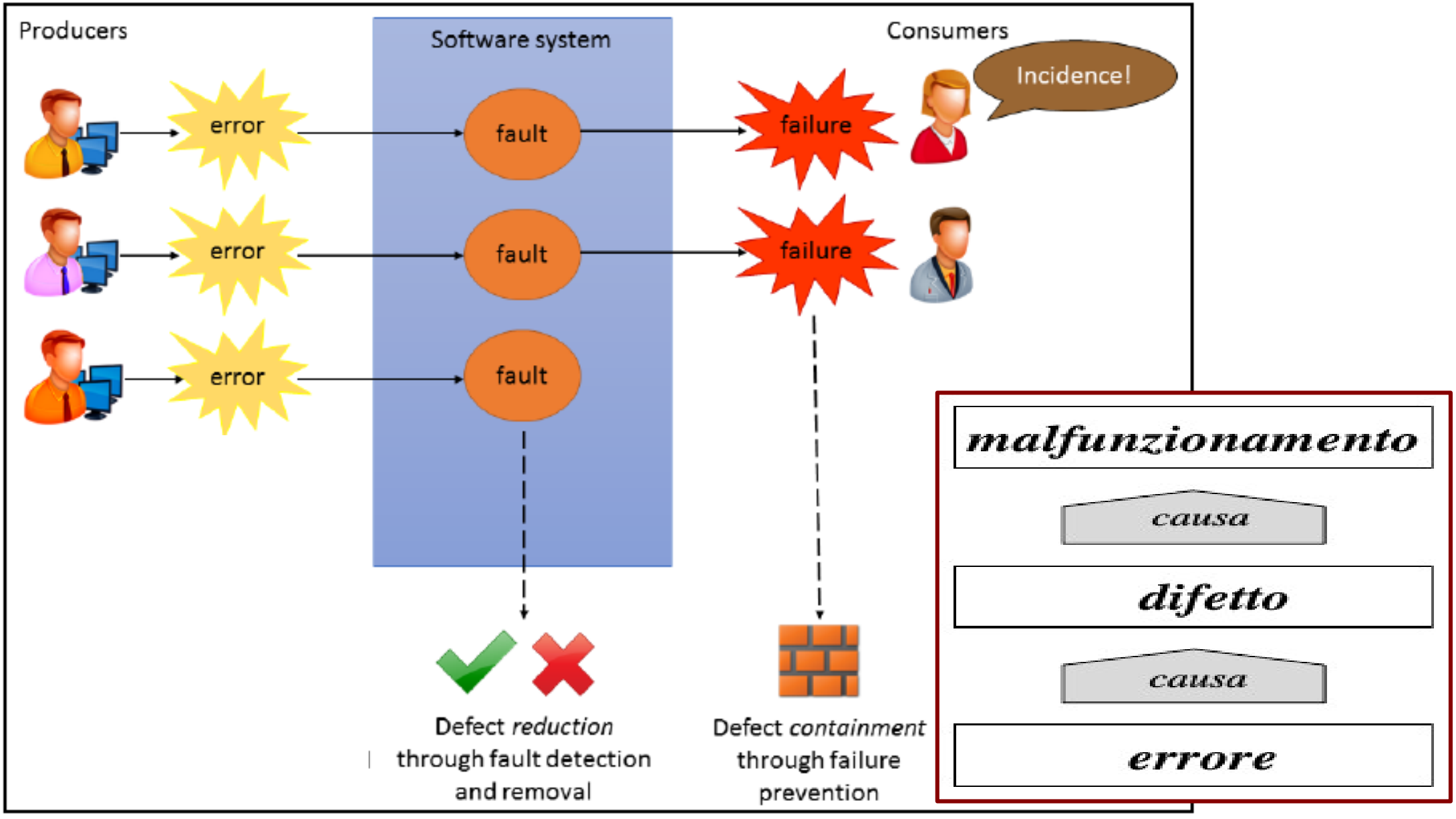


failure prevention

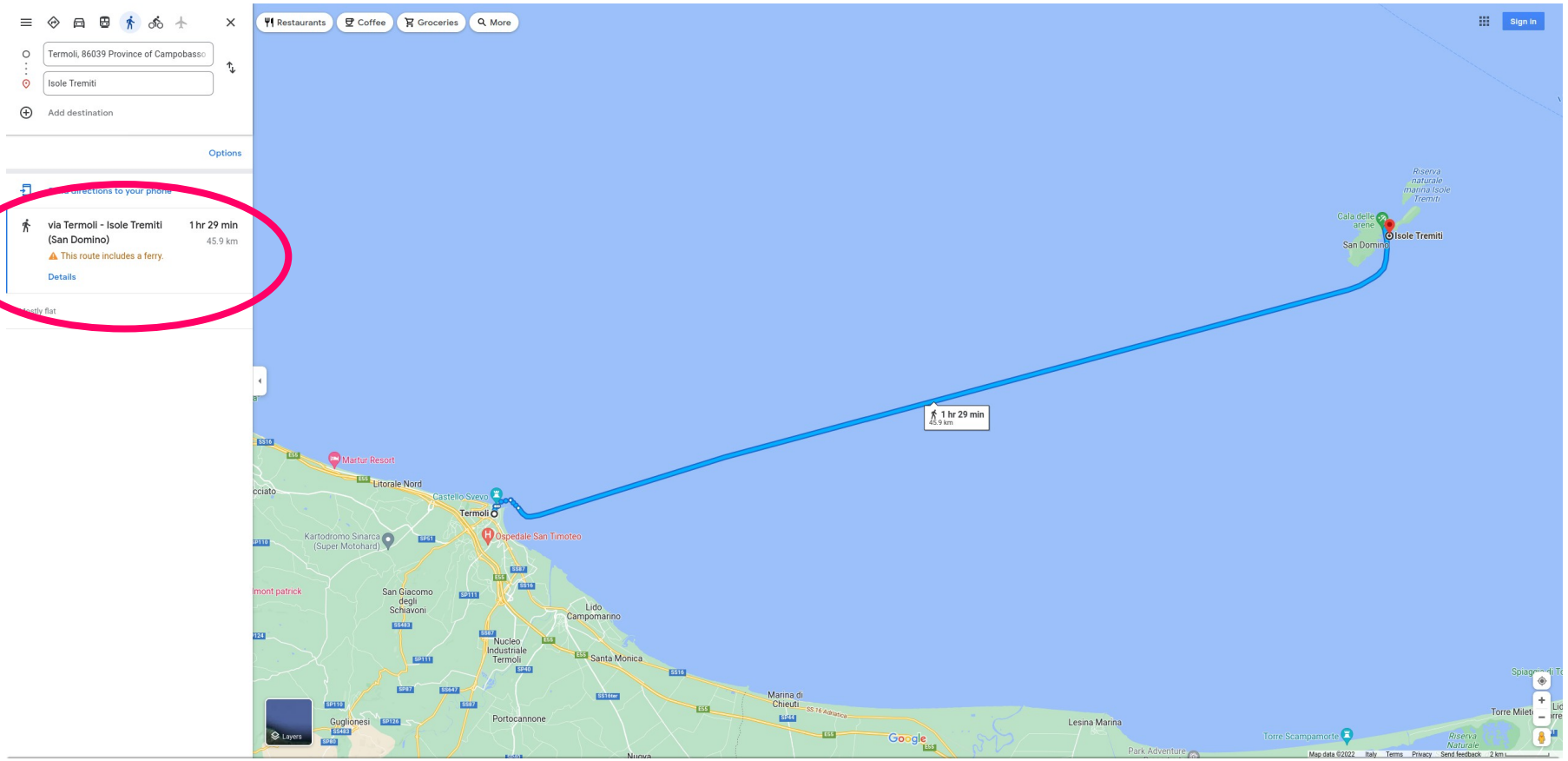


se il punto di partenza è su un isola, allora il sistema di immissione consente di scegliere solo destinazioni nell'isola …. e viceversa

qualche definizione (classica) – 2



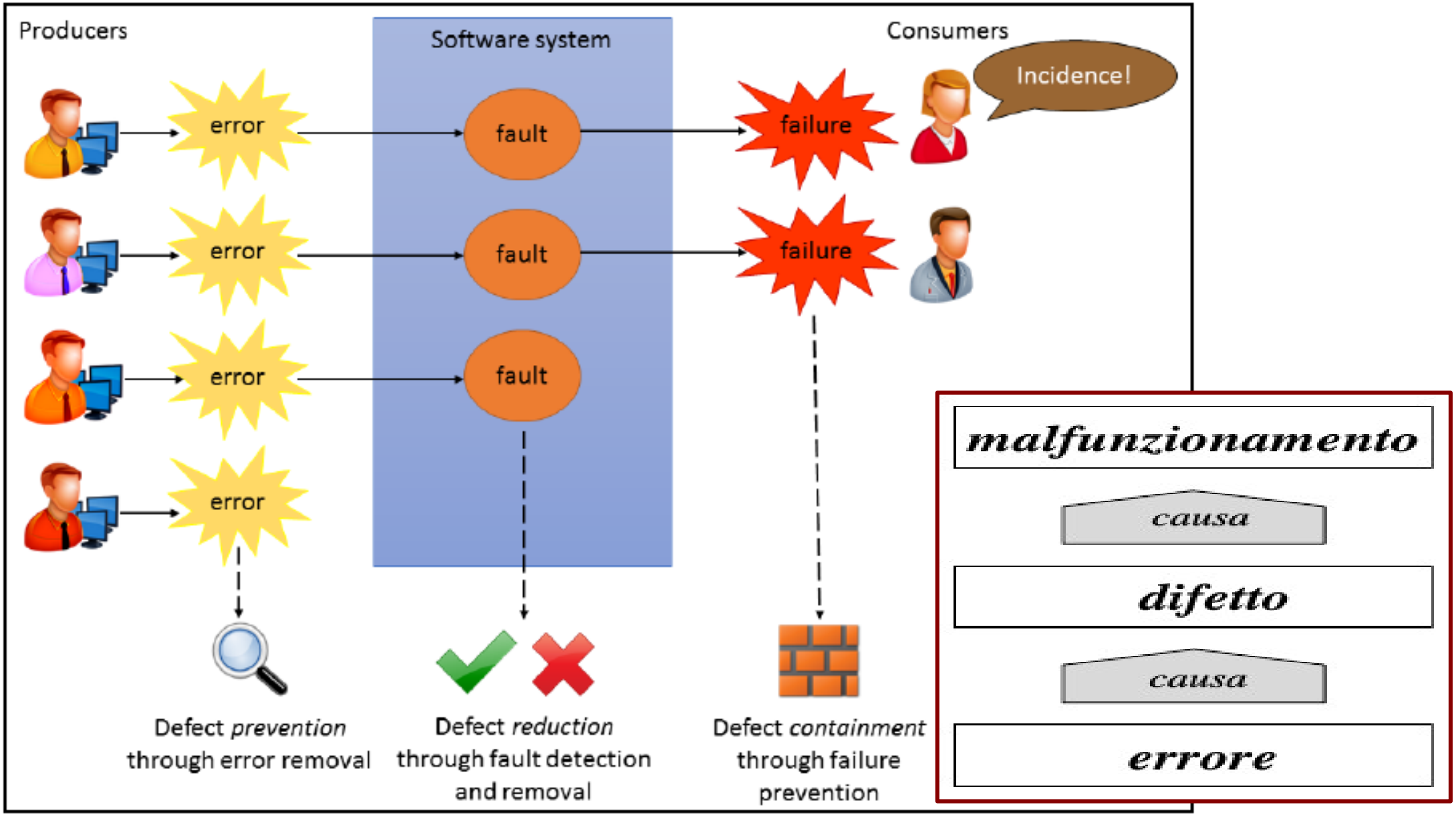
fault detection and removal



il sistema evolve in una nuova versione che corregge il problema

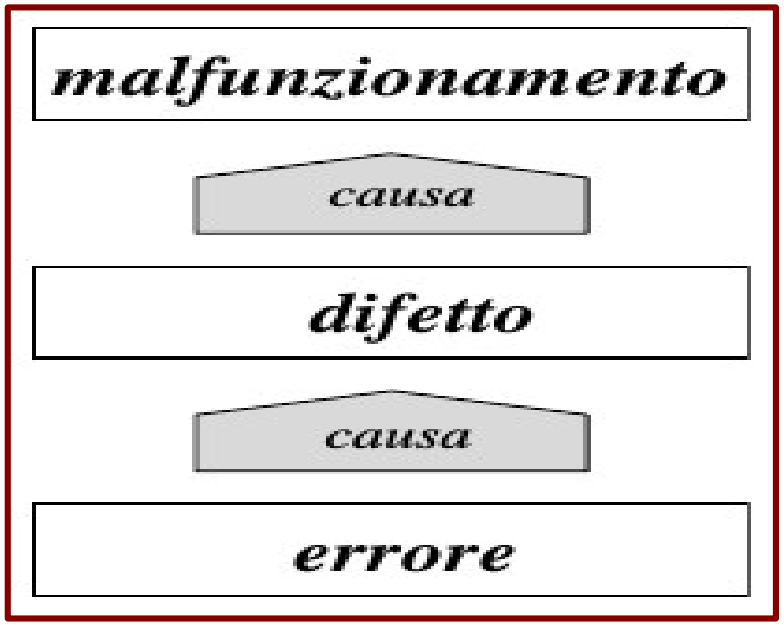
in realtà questa slide presenta un sistema diverso dal precedente. Lo si consideri come una possibile evoluzione del precedente solo a fini didattici per capire il contesto.

qualche definizione (classica) – 2



qualche definizione (classica) – 3

- failure (malfunzionamento) : comportamento non corretto ed osservabile di un sistema software.
  - il concetto di failure è legato al comportamento di un sistema piuttosto che alla sua implementazione specifica in linee di codice
    - il sistema non risponde, risultato atteso diverso, ect.
- fault (bug, difetto): si riferisce al codice/modello
  - condizione necessaria ma non sufficiente al presentarsi di una failure
  - possono essere
    - circostanze non previste
    - azioni non correttamente implementate/modellate
- error (errore): causa di un fault.
  - in genere è un errore umano
    - errore concettuale, di interpretazione, errata digitazione, etc. etc.



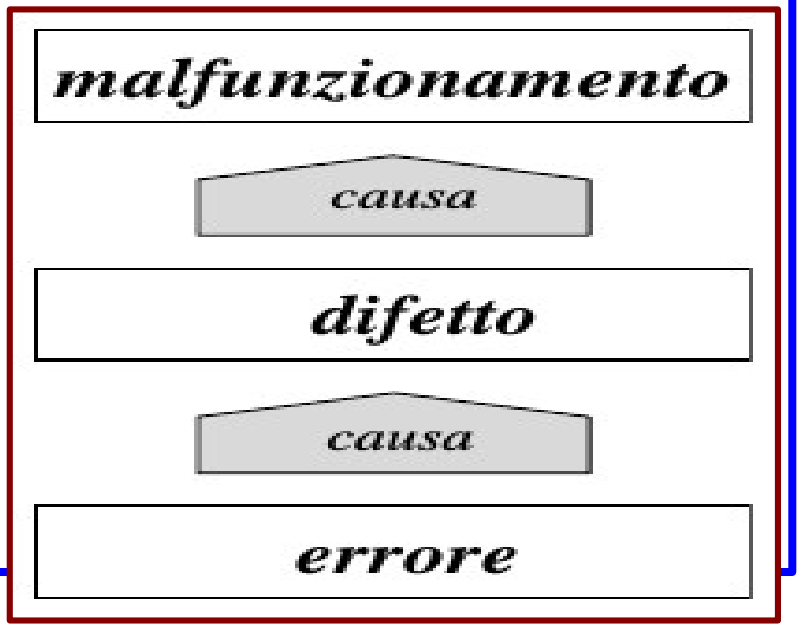
qualche definizione (classica) – 4

Testing

- failure (malfunzionamento) : comportamento non corretto ed osservabile di un sistema software.
  - il concetto di failure è legato al comportamento di un sistema piuttosto che alla sua implementazione specifica in linee di codice
    - il sistema non risponde, risultato atteso diverso, ect.

Debugging

- fault (bug, difetto): si riferisce al codice/modello
  - condizione necessaria ma non sufficiente al presentarsi di una failure
  - possono essere
    - circostanze non previste
    - azioni non correttamente implementate/modellate
- error (errore): causa di un fault.
  - in genere è un errore umano
    - errore concettuale, di interpretazione, errata digitazione, etc. etc.



esempio

```
int makeItDouble(int param) {
   int result;
   result = param * param;
   return(result);
}
```

esempio

```
int makeItDouble(int param) {
   int result;
   result = param * param;
   return(result);
}
```



la chiamata `makeItDouble(3)` torna come risultato `9`

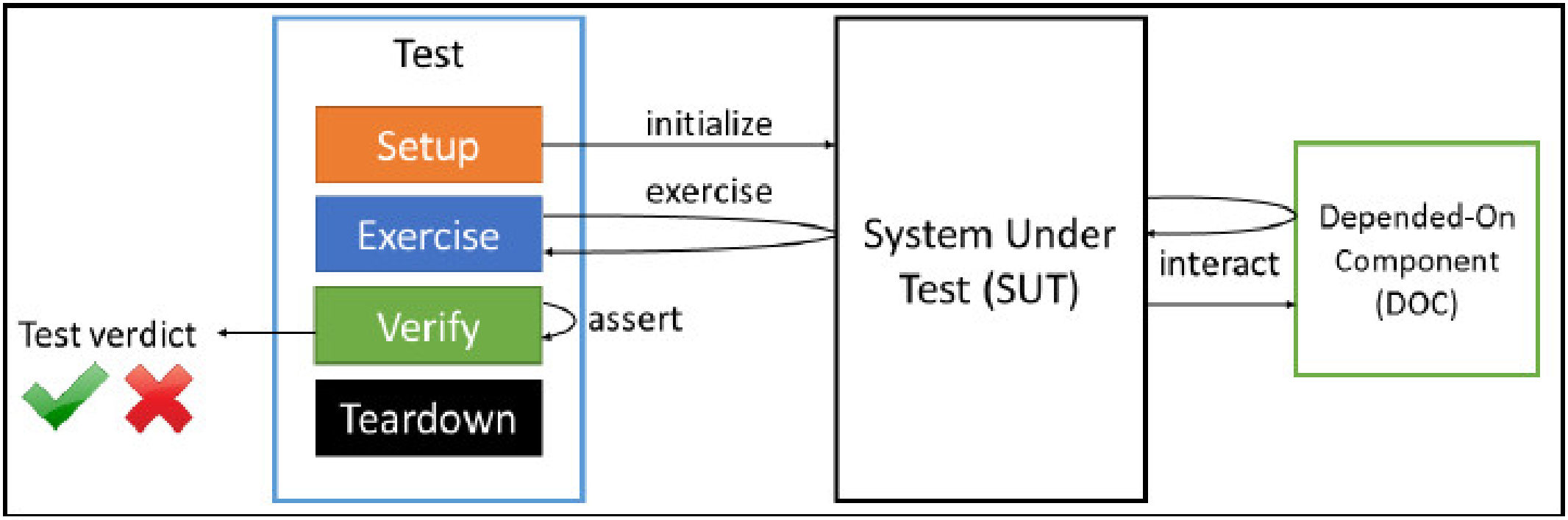
- 9 rappresenta la failure (malfunzionamento)
- questa failure è causata dal fault (difetto) localizzato in 
- possibili fonti per l'errore
  - un typo : ‘`+`’ VS ‘`*`’
  - un errore di interpretazione :
    - `double = param * 2`
    - `power = param * param`

software testing

- esecuzione di alcuni esperimenti in un ambiente controllato al fine di poter acquisire sufficiente fiducia sul suo funzionamento
  - tipicamente è su aspetti funzionali
  - può riguardare anche caratteristiche extra-funzionali

- vantaggi
  - prova se il sistema (reale) risponde alle esigenze per cui era stato creato (avendo almeno un test per ogni requisito)
  - manifesta eventuali malfunzionamenti
- svantaggi
  - *il testing non può dimostrare l'assenza di guasti ma solo la loro presenza* (E.W. Dijkstra)

software testing – contesto tipico



un semplice esempio

Specifica: fattoriale su tutti gli interi. Il fattoriale di un negativo è da intendersi come il fattoriale del suo opposto

un semplice esempio

```
int myBuggyFact(int p){
 int result = abs(p);
 if (result > 1){
   int next = result-1;
   result = result*myBuggyFact(next);
 }
 return result;
}
```

Specifica: fattoriale su tutti gli interi. Il fattoriale di un negativo è da intendersi come il fattoriale del suo opposto

un semplice esempio

```
int myBuggyFact(int p){
 int result = abs(p);
 if (result > 1){
   int next = result-1;
   result = result*myBuggyFact(next);
 }
 return result;
}
```

- supponiamo di voler testare la seguente programma:
  - quanto è ampio il dominio su cui fare test?
  - quanto è costoso esplorare l'intero dominio di input?
  - potrei considerarne una solo una sottoparte?
  - tutti i possibili input sono "equivalenti" tra loro?

Specifica: fattoriale su tutti gli interi. Il fattoriale di un negativo è da intendersi come il fattoriale del suo opposto

un semplice esempio

```
int myBuggyFact(int p){
 int result = abs(p);
 if (result > 1){
   int next = result-1;
   result = result*myBuggyFact(next);
 }
 return result;
}
```

- supponiamo di voler testare la seguente programma:
  - quanto è ampio il dominio su cui fare test?
  - quanto è costoso esplorare l'intero dominio di input?
  - potrei considerarne una solo una sottoparte?
  - tutti i possibili input sono "equivalenti" tra loro?

Specifica: fattoriale su tutti gli interi. Il fattoriale di un negativo è da intendersi come il fattoriale del suo opposto

un semplice esempio

```
int myBuggyFact(int p){
 int result = abs(p);
 if (result > 1){
   int next = result-1;
   result = result*myBuggyFact(next);
 }
 return result;
}
```

- supponiamo di voler testare la seguente programma:
  - quanto è ampio il dominio su cui fare test?
  - quanto è costoso esplorare l'intero dominio di input?
  - potrei considerarne una solo una sottoparte?
  - tutti i possibili input sono "equivalenti" tra loro?

Specifica: fattoriale su tutti gli interi. Il fattoriale di un negativo è da intendersi come il fattoriale del suo opposto

un semplice esempio

```
int myBuggyFact(int p){
 int result = abs(p);
 if (result > 1){
   int next = result-1;
   result = result*myBuggyFact(next);
 }
 return result;
}
```

- supponiamo di voler testare la seguente programma:
  - quanto è ampio il dominio su cui fare test?
  - quanto è costoso esplorare l'intero dominio di input?
  - potrei considerarne una solo una sottoparte?
  - tutti i possibili input sono "equivalenti" tra loro?

Specifica: fattoriale su tutti gli interi. Il fattoriale di un negativo è da intendersi come il fattoriale del suo opposto

un semplice esempio

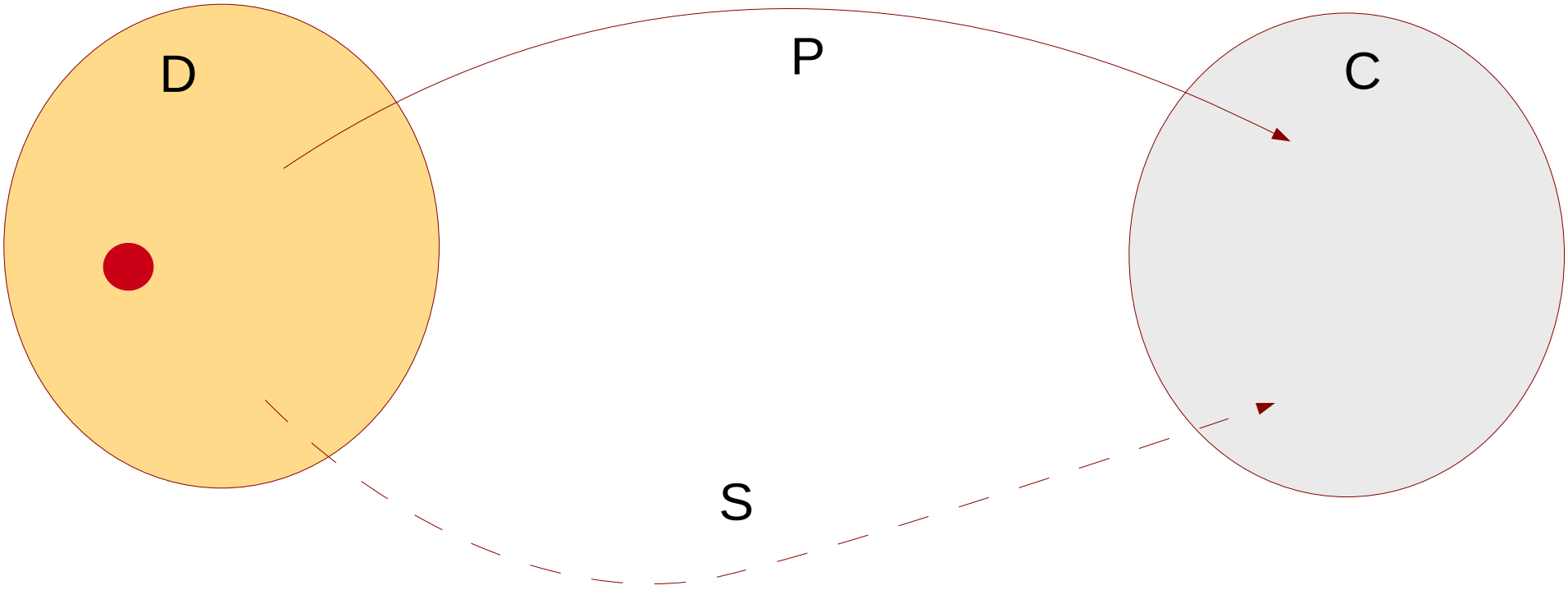
```
int myBuggyFact(int p){
 int result = abs(p);
 if (result > 1){
   int next = result-1;
   result = result*myBuggyFact(next);
 }
 return result;
}
```

- supponiamo di voler testare la seguente programma:
  - quanto è ampio il dominio su cui fare test?
  - quanto è costoso esplorare l'intero dominio di input?
  - potrei considerarne una solo una sottoparte?
  - tutti i possibili input sono "equivalenti" tra loro?

Specifica: fattoriale su tutti gli interi. Il fattoriale di un negativo è da intendersi come il fattoriale del suo opposto

contesto software testing

- in generale D è un insieme molto grande
  - esplorazione esaustiva non praticabile
- selezione attenta delle aree di D per verificare P
  - solo una piccolissima parte di D causa è causa di errori

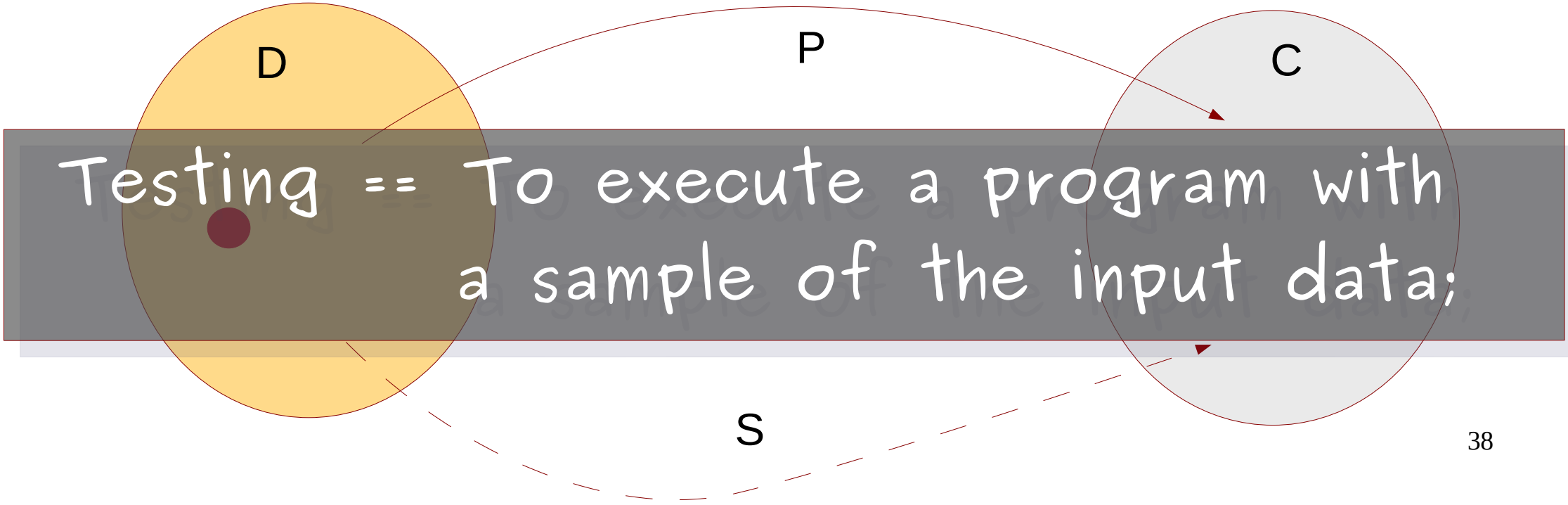


software testing – le domande fondamentali

- quali/quanti input selezionare?
  - scelta di valori rappresentativi
  - partizione del dominio di input ed identificazione dei classi di equivalenza rispetto ad un obiettivo
- quando posso smettere di testare?
  - definizione dei criteri di copertura
    - statement, archi, condizioni, flussi, funzionalità, etc. ect.
  - definizione di valori minimi per l'accettazione
- come faccio a sapere se il risultato di una operazione è corretto?
  - oracle problem
  - si deducono valori attesi da specifiche, storici, sistemi equivalenti in esecuzione, etc. etc

contesto software testing

- in generale D è un insieme molto grande
  - esplorazione esaustiva non praticabile
- selezione attenta delle aree di D per verificare P
  - solo una piccolissima parte di D causa è causa di errori



cosa significa : CORRETTO ?!?!?

- DEFINIZIONE BANALE DI CORRETTEZZA :
  - “corretto” == “senza errori”
- cos

ha ancora senso questa banale nozione di correttezza nel contesto software testing ?!?!?